Write Better Code with Block Diagrams and Flowcharts

by Nathan Jones

Contents

Introduction

Can you imagine trying to assemble a complicated piece of hardware like a 3D printer or a piece of furniture using a set of instructions *that had no pictures in it*? You open the manual, full of excitement, and are greeted by nothing but a wall of text.

"Insert screw A into slot B."

"Attach flange G to partition BB, making sure not to obstruct the opening in wall EE."

His instructions must have had pictures on them.

What a mess that would be! And yet, we so often think that we can write or read code without such graphical aids. Over the last four years of teaching at the university level, I've wondered about the concept of expertise and about what skills help me to solve problems so much faster or better than my students. In my estimation, a prevailing reason is that *I am more ready and able to draw an appropriate sketch of the problem at hand* than they are, and it is that sketch which helps me both define the problem and identify any omissions or inconsistencies in my understanding.

In this article, I want to share with you two such graphical aids, the **software block diagram** and the **flowchart**, that can help you not only write better code but also more quickly understand someone else's code. These diagrams provide an architectural or high-level depiction of your system, which has many benefits in both scenarios.

When writing code, an architectural view of the system helps you to think through the *problem* before you dive into any one *solution*. Before writing a thousand lines of beautiful code it would probably be good to know that the code you're writing will actually solve the problem at hand! Additionally, by thinking through the general structure of the solution before actually implementing anything, you encourage your design to be more modular. For decades, studies have proven over and over again that modular code is easier to design, write, test, and debug.

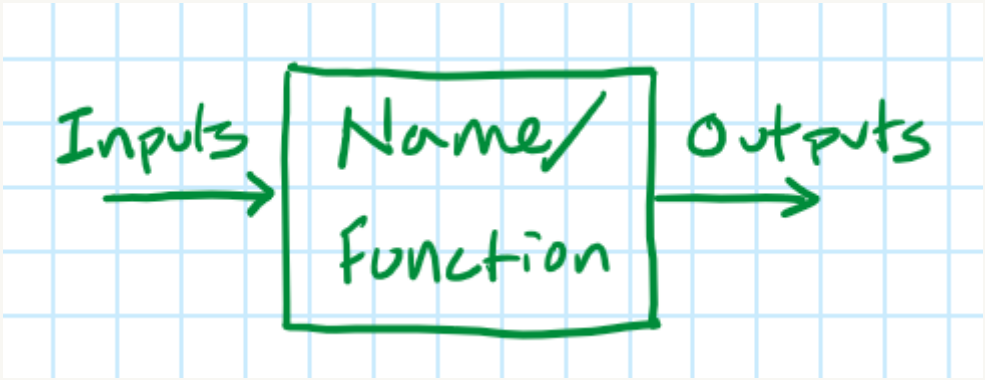
When reading code, an architectural view of the system helps you understand how each class, function, and variable fits into the larger picture of the application code. The human brain has only so much working memory and at some point (after 50 LOC, 100 LOC, 1000 LOC, etc) even code with excellently named functions and verbose comments can become inscrutable without a roadmap to keep you oriented to how the code you're reading fits into the larger picture.

Alright, let's get started!

Software Block Diagrams

What is it?

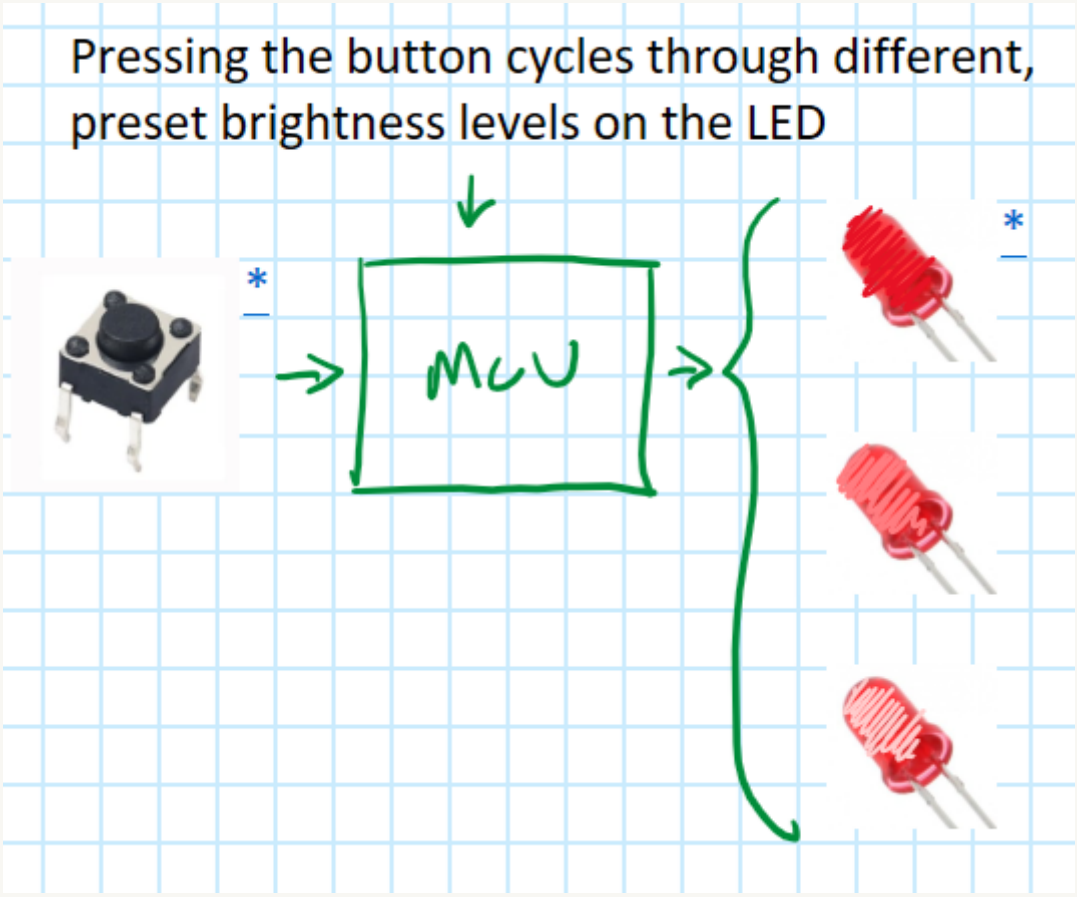
A software block diagram depicts a system using (1) blocks that perform tasks (transforming some kind of input into some kind of output) and (2) links that connect one block's output to another's input. By linking the blocks together, we can compose an entire system.



A software block diagram is the same as a functional block diagram and it's similar to a data flow diagram. Jacob Beningo described this as the "Outside-In" approach to RTOS design in his 2021 Embedded Online Conference talk "Best Practices for RTOS Application Design" (starting at 10:45).

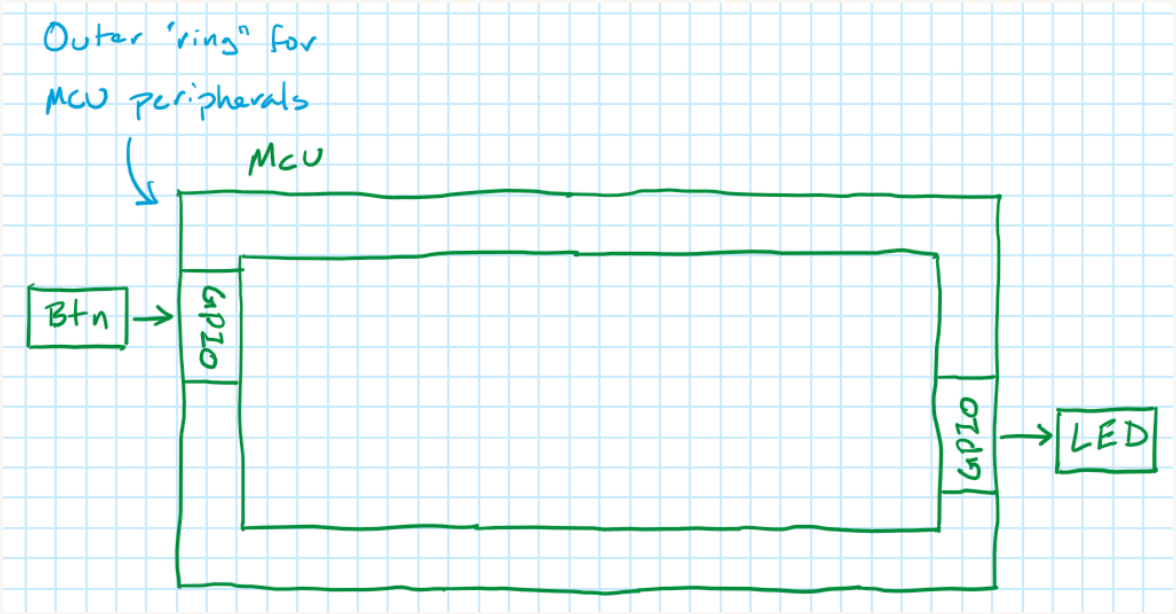
How do I draw one?

By way of example, let's pretend that we're trying to build a simple system: a button will be used to cycle through various brightness levels on an LED. Each button press should increase the brightness up to some maximum value and then roll back over to the starting value. PWM waves with different duty cycles will be used to create the different brightness levels on the LED.



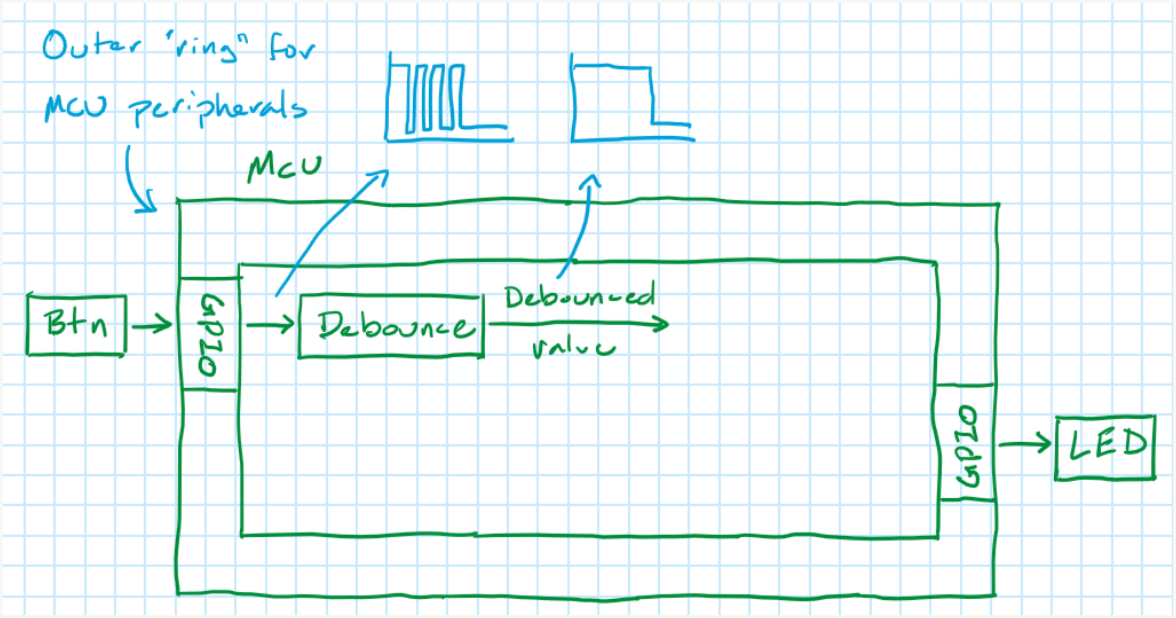
To begin with, we'll draw something that looks like a picture frame to represent our microcontroller. Things inside the frame represent software components and things outside the frame represent hardware components. Inside the border of the frame is where we'll indicate which microcontroller peripherals we'll use to interact with the hardware.

In our case, our two pieces of hardware are the button and the LED. Both will interact with the GPIO peripheral of the microcontroller.

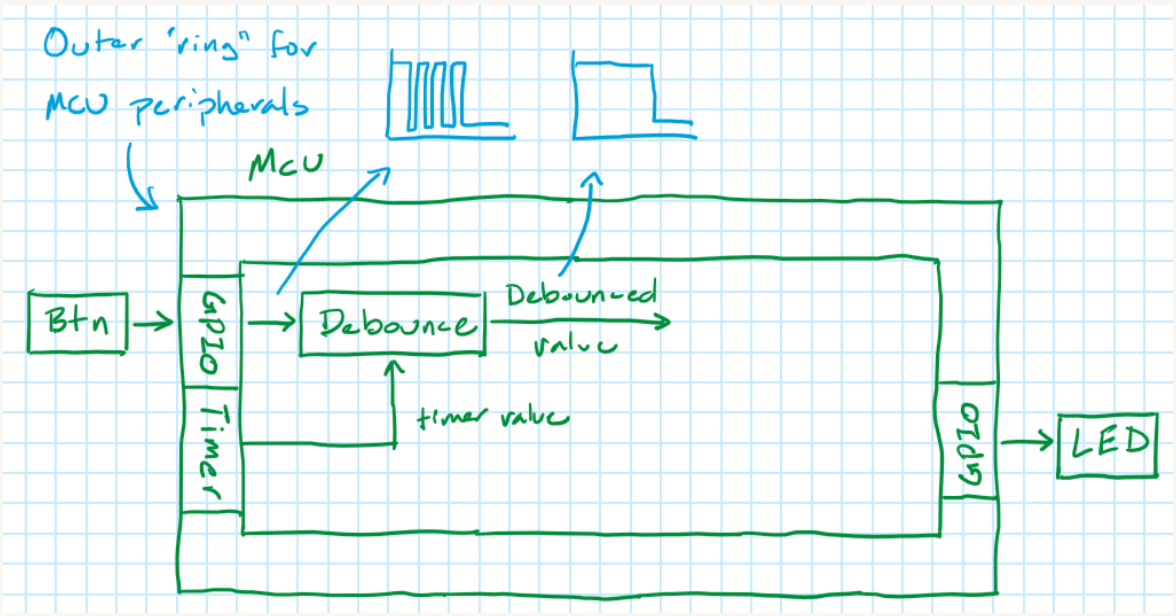


At this point, we can iteratively (and recursively) ask, "What happens to the input values in order for them to become the appropriate output values?"

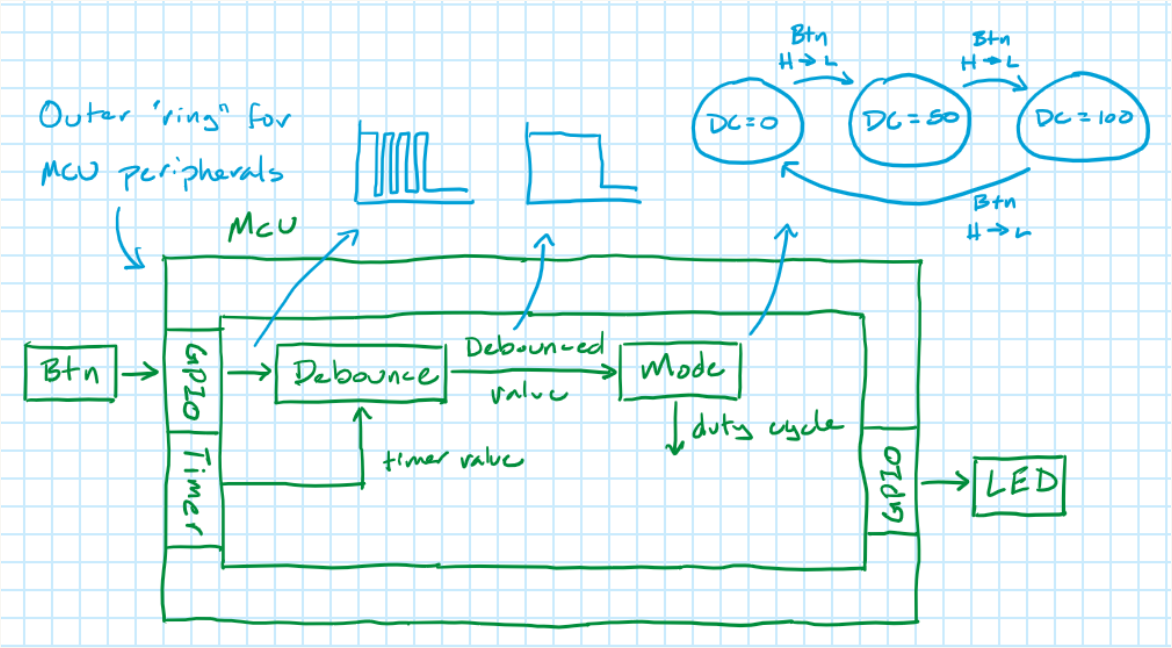
The button likely bounces so we can reasonably assume that the first part of the system will need to take in the noisy button signal and output a debounced button signal.



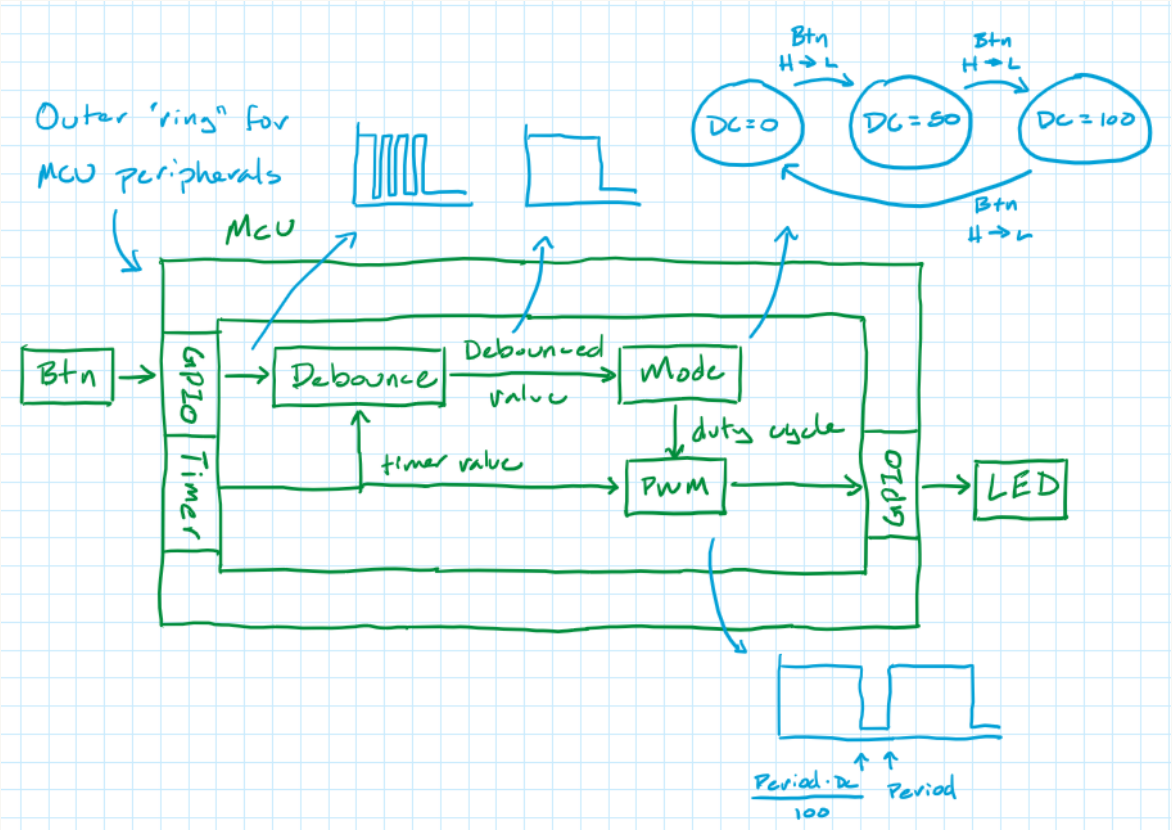
Most software debouncing techniques require polling the button GPIO pin at regular intervals to provide an accurate debounced value, so let's also add a "Timer" peripheral to the MCU border and a connection from that to the "debounce" block.



A high-to-low transition on the debounced button signal indicates that the button was pressed and should result in a change to the PWM signal's duty cycle. So, next, we'll place a block that takes the debounced button signal as an input and converts it into a value for the duty cycle. For simplicity, we'll say that the LED defaults to off (0% duty cycle) and pressing the button cycles it through 50% and 100% duty cycles before returning to 0%.



Lastly, we'll need a block to generate the PWM wave based on the value of the duty cycle set by the previous block; we'll also assume that the constant value `period` was initialized at the top of our program. This block will also need to be aware of the passage of time, so we'll make a connection between it and the "Timer" peripheral. The output of this block goes directly to the GPIO pin controlling the LED.



Our block diagram is now complete! Even though this was a simple example, I hope that you can see that we have a much clearer path for implementing our code than we did when we started, when we had only a basic description of the project requirements to go off of.

At this point I want to make a few important observations:

- If you have **more than a dozen or so blocks** in your diagram, you're putting too much detail into this top-level drawing. The purpose of this first block diagram should be to provide a single, high-level overview of your system ***which can fit on a single page of paper***. This is critical to being able to understand the system! Anything more and it starts to get difficult to see the forest for the trees.
    - For any blocks that you can't envision how to write in your head (such as something really complicated like "Extract critical features from input stream" or "Update UI"), grab a second piece of paper and draw the block diagrams for those blocks there (that's why I said above that we might *recursively* ask the question "How do these inputs become these outputs?"). Continue drawing nested block diagrams until you can write out blocks like those above that seem relatively easy to write out in code.
- Other than what's been defined by the project, **there are no implementation details on this block diagram**. There's nothing in our diagram that *requires* our block to be something like a main loop task or an interrupt, neither is there anything that *requires* the debounced button or duty cycle values to be inserted into a queue or written to a global variable. This is intentional since the block diagram is meant to be an architectural view of the system; if implementation details had leaked into this diagram then we might find out later on that our design was overly brittle by being tied to those details.

In summary, the steps for drawing a software block diagram are:

1. Draw the MCU "frame" and any external hardware components.
2. Identify the 6-12 high-level operations your system must perform and link them together by connecting the output of each block to the input of one (or more) blocks.
3. For any block that doesn't seem simple enough that you can envision in your head what the code might look like, get a second piece of paper and draw another block diagram for that specific block (continuing as needed).

Implementing a block diagram in code

Now that we have an idea of what the system needs to do in order to fulfill the design requirements, we can start to add some implementation details. A key observation for this step is that **blocks perform tasks** and then **communicate their outputs to other blocks by writing a value to memory or to a processor register**.

Thus, each block represents something the MCU actually *does*. We can accomplish this in a number of ways, such as:

- making each block a main loop or RTOS task,
- running each block inside an appropriate interrupt,
- making a block a standalone function that gets called by a task or interrupt, or
- assigning a block to a hardware peripheral (to be executed asynchronously to the normal program flow).

For example, the "debounce button" block could be run inside an RTOS task that polls the button GPIO pin at regular intervals:

```
void taskDebounce(void)
{
    // Read button GPIO pin
    // After 10 successive reads of the same value:
        // Post new button value
    // Delay 10 ms
}
```

Or maybe that block is put inside the timer interrupt:

```
// Timer ISR is set to trigger every 10 ms
void timerISR(void)
{
    // Read button GPIO pin
    // After 10 successive reads of the same value:
        // Post new button value
}
```

Additionally, we may find in doing this that certain blocks can be combined into the same task or interrupt. The "debounce button" and "mode" blocks may be so closely tied together that it makes more sense to have a single task that reads the button GPIO pin and outputs the current duty cycle value than it does to keep those pieces of code separate.

Next, each arrow represents some value that another block (i.e. a task or interrupt) reads from. We can also accomplish this in a number of ways, such as:

- writing to and reading from a global variable,
- writing to a local variable via a "setter" function,
- placing a value in a queue,
- setting a flag to alert the consuming task that a new value is ready (and also providing a "getter" function for that task to call when it next runs to retrieve the new variable),
- etc.

For example, the "duty cycle" value could be a global variable:

```
uint8_t g_dutyCycle = 0;
void mode(void)
{
    // On button press event:
    if (g_dutyCycle == 0) g_dutyCycle = 50;
    else if (g_dutyCycle == 50) g_dutyCycle = 100;
    else g_dutyCycle = 0;
}
```

```
void PWM(void)
{
    // If timer value is less than g_dutyCycle: LED on
    // Else LED off
}
```

Or, if we wanted to be fancier, we could put it into a queue from which the PWM task could read:

```
void mode(void)
{
    // On button press event:
    if (g_dutyCycle == 0) enqueue(queue_PWM, 50);
    else if (g_dutyCycle == 50) enqueue(queue_PWM, 100);
    else enqueue(queue_PWM, 0);
}
void PWM(void)
{
    static uint8_t dutyCycle = 0;
    if queueIsNotEmpty(queue_PWM) dutyCycle = dequeue(queue_PWM);    // Get new
value
    // If timer value is less than dutyCycle: LED on                // Control
LED
    // Else LED off
}
```

Each of these has various design trade-offs, leaving you plenty of options to implement your block diagram in the way you see fit.

Ready for a small challenge?

See if you can modify the block diagram above to accommodate the following scenarios! How would the block diagram change (if at all) if you wanted:

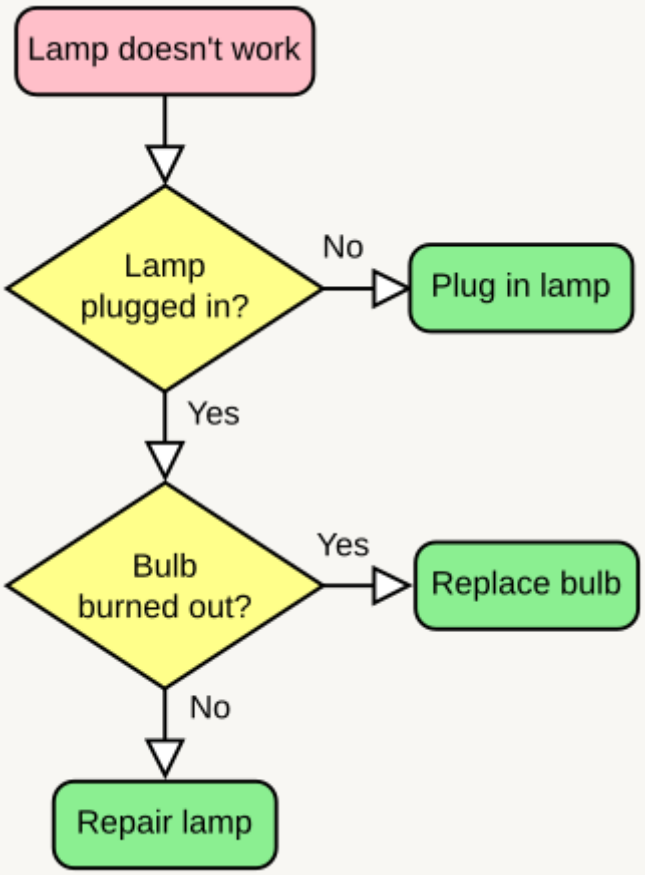
- a long press on the button to cycle through each brightness setting automatically while the button is held down?
- to use a second button to control the PWM frequency?
- the LED to fade up or down when the brightness was changed (as opposed to changing the duty cycle instantaneously)?

Flowcharts

What are they?

A flowchart depicts the steps in a process using essentially two types of blocks: (1) rectangles (typically) that execute software tasks and (2) diamonds (typically) that evaluate a condition (usually binary) and take one of two or more branches based on the value of the condition.

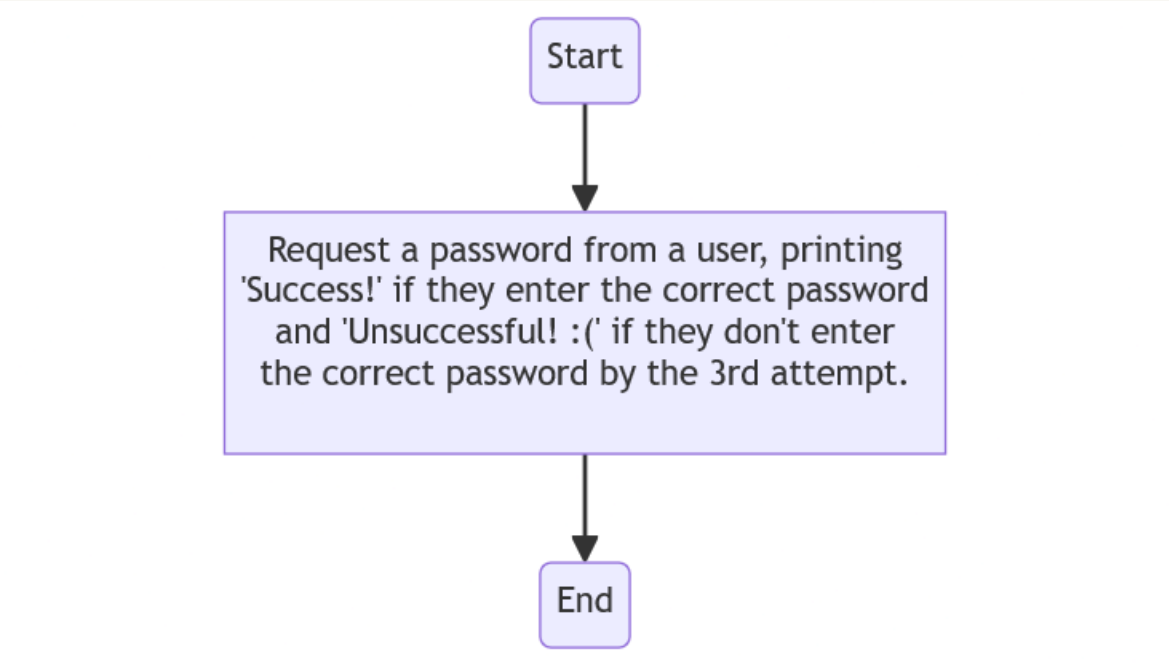
(Technically, the flowchart may also include a "start" and "end" block, which are typically ovals.) The flowcharts we'll look at today are purely sequential, although UML flowcharts (called "activity diagrams") include symbols that allow for the representation of multiple threads, which then join back together at some point later on.



It's hopefully not a stretch to see that with just those two elements (effectively, statements and branches) a flowchart is able to represent any control flow element (such as conditionals, `switch...case` statements, `for` loops, `while` loops, etc) in any imperative programming language such as C, C++, Java, or Python. In essence, this means that we can use a flowchart to depict any piece of procedural code we might write in those languages, which is all of the code that would appear inside a function. (We obviously can't use flowcharts to create something like classes, but we can clearly use flowcharts to depict how we might write the code *inside* a class's methods.) This makes them ideal for depicting how each of the blocks in our block diagram (i.e. the tasks in our system) should be implemented.

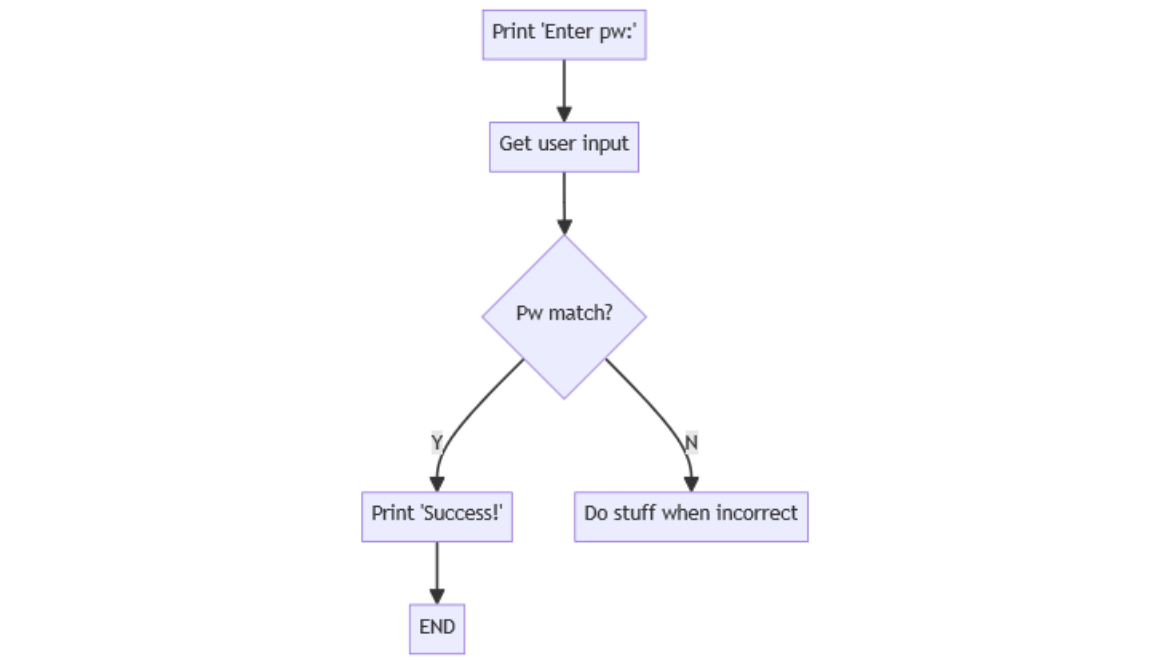
How do I draw one?

By way of example, let's pretend that we're trying to write a simple piece of code. The code will request a password from a user, printing "Success!" if they enter the correct password and "Unsuccessful! :(" if they don't enter the correct password by the 3rd attempt.



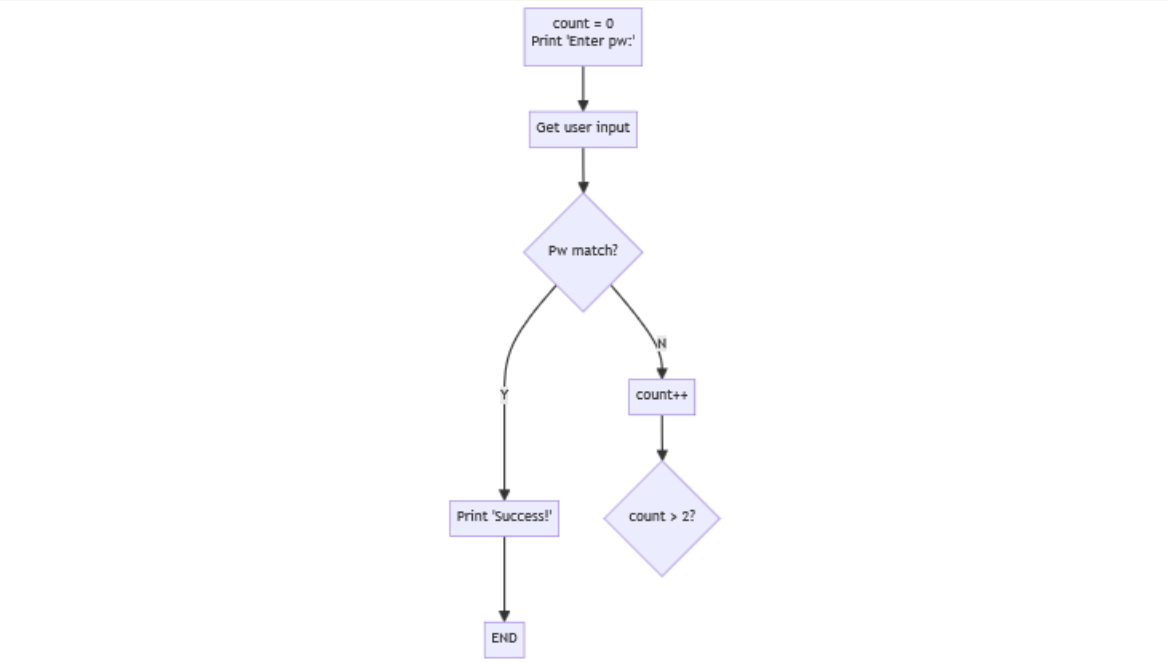
The best way to create a flowchart is to use a process called **sequential decomposition**, which just means that we start with a high-level description of our function (like in the diagram above) and then continue breaking it down into smaller and smaller pieces until every element in our flowchart can be readily implemented in actual code. (An alternate method would be to work through the flowchart sequentially or chronologically, the same way it would be executed on our microcontroller.)

Our example function hinges around getting and checking a piece of user input, so let's start by breaking up our mega-task above into that one conditional.

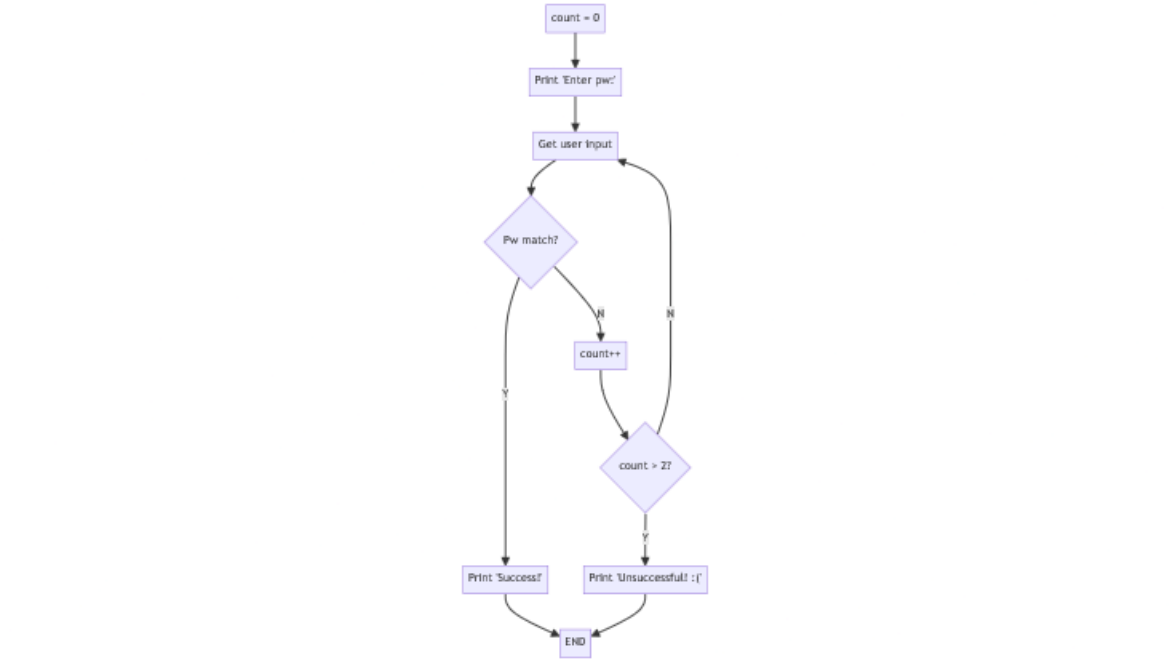


I've separated the `Print 'Enter pw:'` and `Get user input` blocks because I *think* I'm going to loop back around and I won't want to repeat that first block when I do.

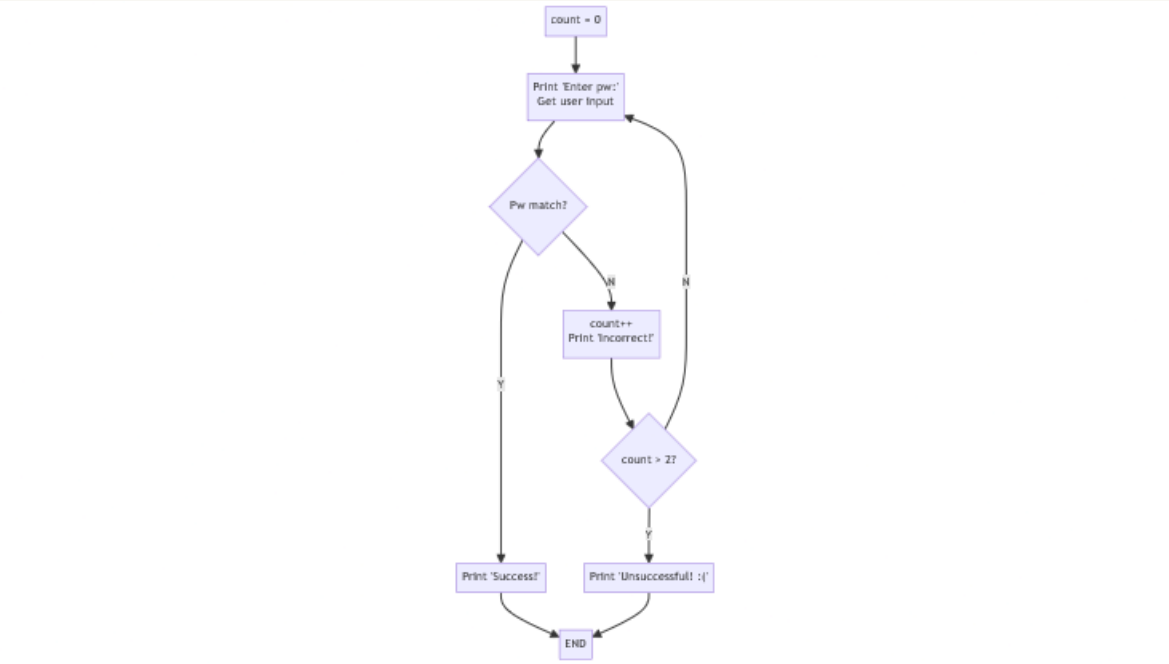
Okay, the "happy path" (when the user enters the correct password) is so easy that it's done already. Let's take a closer look at the "unhappy path". If the user's input *doesn't* match our password, we need to prompt them again. But wait! We need to do something different if this is their *third* incorrect attempt. Oh yeah, so we need to initialize a variable to keep track of those incorrect attempts and then increment that variable if they do guess incorrectly.



If the number of unsuccessful attempts is less than 2, we'll prompt the user again. Otherwise, we'll print the failure message.



And now that I'm thinking about it, I guess we *do* want to print "Enter pw:" on every attempt (lest the user just be looking at a blinking cursor), so let's combine that block with the one below it. Also, let's print a message to let them know that their entry was incorrect if it was.



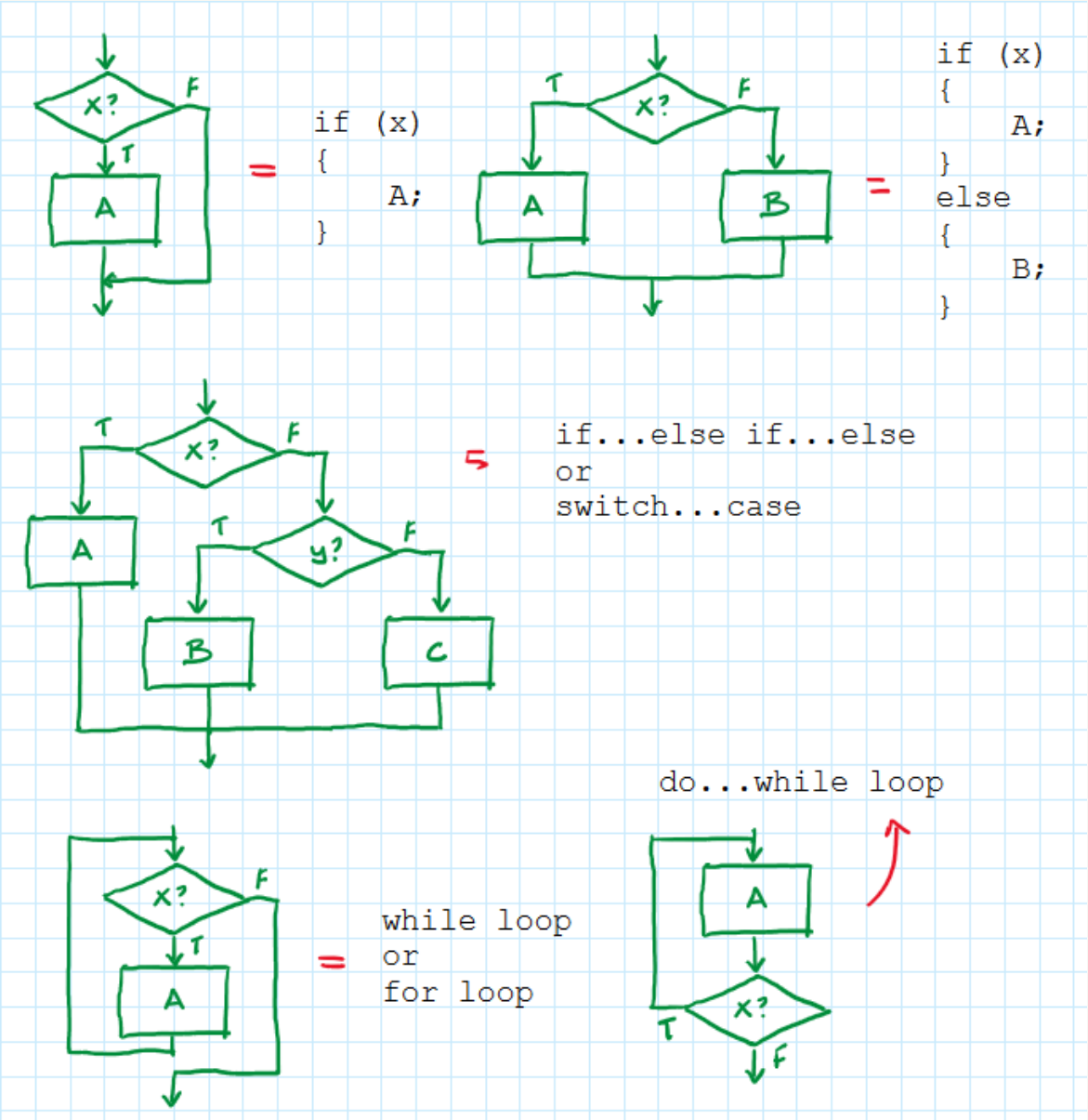
Again, this is a simple example, but I hope you can see that iterating over a flowchart helped us literally *see* the code as we were designing it, which made it so much easier to make sure each line of our code was in the right spot. Along the way we were able to perform some simple refactorings and visually run through the "unhappy" paths to ensure that our function could properly handle edge cases and erroneous inputs.

In summary, the steps for drawing a flowchart are:

1. Draw a single block which represents the entire task that is to be completed by your flowchart.
2. Break up that task into smaller tasks, one at a time, until each flowchart element can be readily implemented in actual code.

Implementing a flowchart in code

At first pass, you might think, "Shouldn't this be easy? I just match the parts of my flowchart to one of the elements below and write out the resulting code. Done."



For flowcharts on the simpler side, you'd be completely correct! May your flowcharts always be so simple.

However, even our little flowchart above for the password checker throws a wrench in this simple algorithm: although it *looks* like the main flowchart could be implemented with a `for` or `while` loop, we have a branch to the "Success!" message and the end of the flowchart (the "happy path") in the middle of what otherwise looks like a simple loop. There are several ways to solve this problem, each involving different and/or creative uses of the C/C++ language.

Solution #1: Use `break` to jump to the bottom of the loop

```
uint8_t count = 0;
char * password = "OpenSesame";
do
{
    char guess[32] = {0};
    printf("Enter pw: ");
    scanf("%s", guess);
```

```
    if (strcmp(guess, password) == 0)
    {
        printf("Success!\n");
        break;                              // Breaks out of do...while loop to
end
    }
    else
    {
        count++;
        printf("Incorrect!\n");
        if (count > 2)
        {
            printf("Unsuccessful! :(\n");
            break;                          // Breaks out of do...while loop to
end
        }
    }
} while !(count > 2);
```

Solution #2a: Use a flag variable (`success`) to keep track of system status

```
uint8_t count = 0;
char * password = "OpenSesame";
bool success = false;
while ( (count < 3) && !success )
{
    char guess[32] = {0};
    printf("Enter pw: ");
    scanf("%s", guess);
    if (strcmp(guess, password) == 0) success = true;
    else
    {
        count++;
        printf("Incorrect!\n");
    }
}
if (success) printf("Success!\n");
else printf("Unsuccessful! :(\n");
```

Solution #2b: Use a `for` loop instead of a `while` loop to keep track of the count variable

```
char * password = "OpenSesame";
bool success = false;
for (uint8_t count = 0; (count < 3) && !success; count++)
{
    char guess[32] = {0};
```

```
    printf("Enter pw: ");
    scanf("%s", guess);
    if (strcmp(guess, password) == 0) success = true;
    else printf("Incorrect!\n");
}
if (success) printf("Success!\n");
else printf("Unsuccessful! :(\n");
```

This one doesn't really solve the problem of our flowchart any differently, but it does show the equivalence of `for` and `while` loops and also happens to be the most condensed solution we've seen so far!

Solution #3: Replace "jump to end" with early function return

```
char * password = "OpenSesame";
void checkPassword(void)
{
    uint8_t count = 0;
    while(1)
    {
        char guess[32] = {0};
        printf("Enter pw: ");
        scanf("%s", guess);
        if (strcmp(guess, password) == 0)
        {
            printf("Success!\n");
            return;
        }
        else
        {
            printf("Incorrect!\n");
            if (++count > 2)
            {
                printf("Unsuccessful! :(\n");
                return;
            }
        }
    }
}
checkPassword();
```

You can even do weird things like intermingle switch...case statements with do..while loops, such as in the infamous "Duff's device"! It turns out, though, that a general solution to this problem is really hard. The fancy name for "flowchart" is "control flow graph" and another term for the executable code that we're trying to turn them into is pieces of "structured programming". There's been lots of interesting research into the problem of turning control flow graphs into

pieces of structured programming, some even that suggests that it's *not possible* to do for *all* control flow graphs or that's it's not possible without using `goto`s.

Ready for a small challenge?

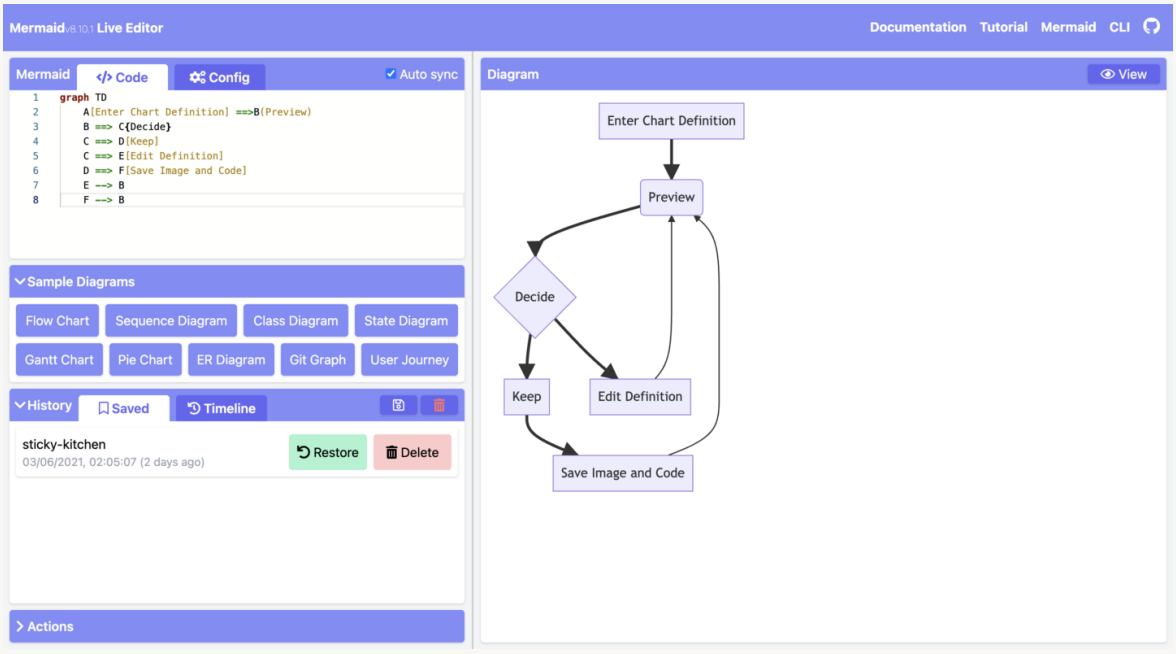
See if you can modify the flowchart above to accommodate the following scenarios! How would the flowchart change if you wanted:

- each incorrect attempt to result in a lengthening delay before the user could re-attempt their password again?
- to provide a "forgot password" feature that asks the user 3 predefined security questions and prints "Success!" if they can correctly answer all on their first attempt?

Neat tools for creating software block diagrams

Mermaid

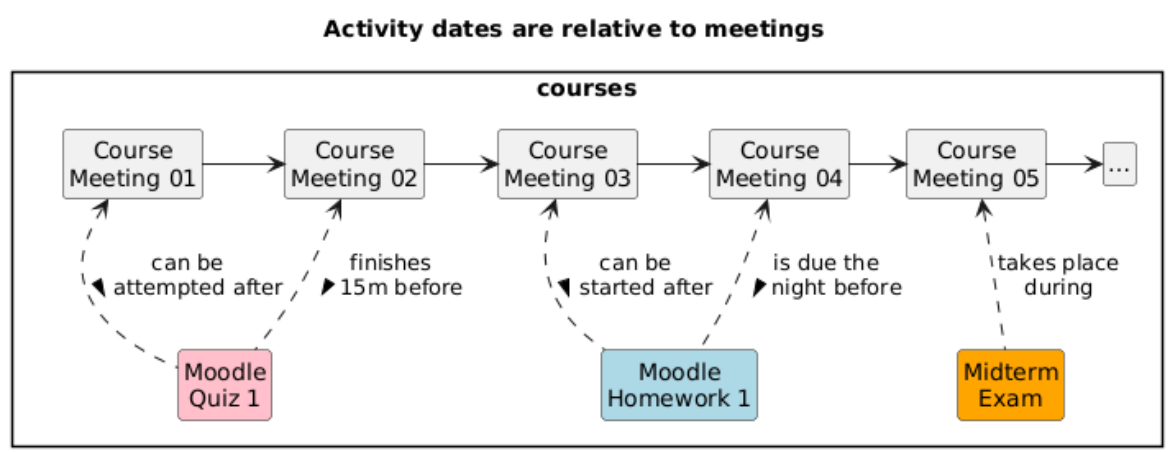
Mermaid is a "JavaScript based diagramming and charting tool that renders Markdown-inspired text definitions to create and modify diagrams dynamically." Using their live editor allows you to instantly see the effect of your text descriptions (below left) on the rendered diagram (below right). Additionally, GitHub supports the rendering of Mermaid diagrams in your Markdown files.



Mermaid supports many different types of drawings, including block diagrams and flowcharts. The flowcharts above were created using Mermaid.

PlantUML

PlantUML is another "text-to-graphic" rendering tool that you can use to build all kinds of UML (and non-UML) diagrams, including block diagrams (what they call "component diagrams") and flowcharts (called "activity diagrams"). Like Mermaid, PlantUML also provides an online editor.

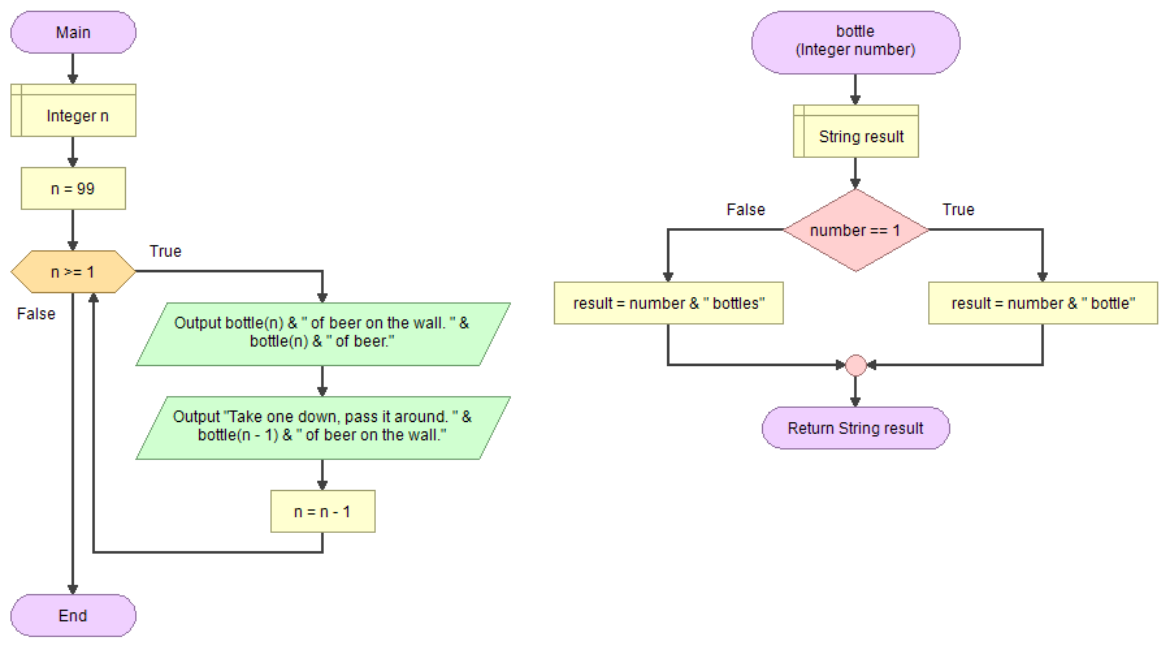


Example PlantUML block diagram, rendered with the online editor from this PlantUML description.

GitHub integration seems possible, though more difficult than with Mermaid.

Flowgorithm

Flowgorithm is a simple graphical tool for creating flowcharts. That's about all that the software does so the interface is quite simple; adding an element is as easy as right-clicking on an arrow and selecting the type of block to insert in between the flowchart elements that were previously connected by that arrow.

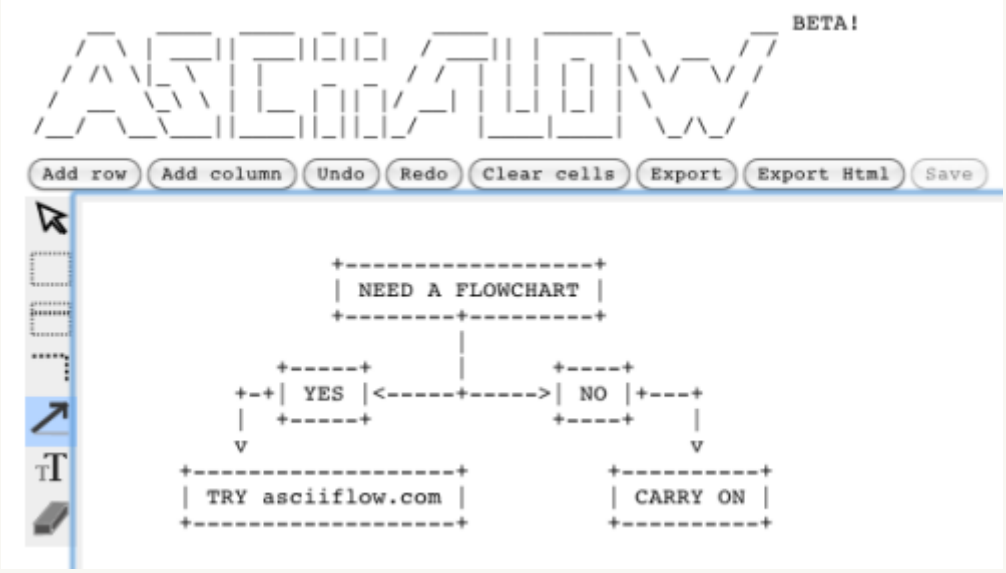


That being said, the software does have several neat features. The *coolest* one, in my opinion, is that Flowgorithm will automatically generate code from any flowchart you create for any one of 29 different programming languages, including C++, Bash, Ruby, Swift, Python, and Java.

Additionally, Flowgorithm has support for Turtle graphics, file IO, and a console (so you can actually run/test your flowcharts as you are writing them). Flowgorithm also provides some basic debugging features in the form of a variable watch window and conditional breakpoints.

ASCIIflow

ASCIIflow is a *very* basic drafting tool that lets you draw boxes and arrows and make simple annotations.



Making diagrams with this tool isn't exactly easy (and they will definitely be harder than the tools above to edit later), but the really neat thing about this tool is that you can export your drawing into standard ASCII/Unicode characters, meaning that you can *embed your drawings directly into your code as a block comment*.

```
/****************************************
      Flowchart for function foo
                ┌─────────────────┐
                │NEED A FLOWCHART│
                └─────────────────┘
                         │
                         │
                ┌─┐      │      ┌─┐
           ┌────│Y│◄─────┴─────►│N│────┐
           │    └─┘             └─┘    │
           ▼                           ▼
┌──────────────────┐          ┌──────────┐
│TRY asciiflow.com│          │CARRY ON│
└──────────────────┘          └──────────┘
```

```
************************************/
void foo(void){...}
```

This can help make sure that your drawings stay consistent with any of your code changes, since the two things live in the same file.

Why not use LucidChart, Draw.io, Visio, etc?

With the exception of ASCIIflow, I'm not a fan of general drafting programs such as those listed in the heading above. My problem (and I completely realize that it's a personal one) is that it's too tempting to make every iteration of my diagrams look pretty if I'm using a general drafting tool and once there are more than a half-dozen elements in my diagram, making sure the diagram is readable and pretty after every alteration can take a not-insignificant amount of time. If my documentation is onerous to edit then I'm not likely to edit it when it changes and the chances of it becoming inconsistent with my code (or worse, dead and forgotten) are much higher. Personally, I prefer to live with drawings that I don't have *full* control over (since the rendering tools do that for me) in exchange for drawings that are easy to create and edit.

Conclusion

Diagrams are an important part of writing and reading code well; akin to having diagrams or graphics in a set of complex assembly instructions for a 3D printer or a piece of furniture. If you *don't* have a diagram to help describe your code as you're writing or reading it, you're doing yourself a disservice!

When writing code, a **block diagram** can help you create a program structure that actually solves the problem at hand (instead of whatever you *think* the problem is). Additionally, by default it encourages a modular design, which will produce code that is easier to design, write, test, and debug. Once drawn, you can determine how to implement your tasks (as main loop/ RTOS tasks, interrupts, etc) and the means by which your tasks communicate their output values (through globals, a queue, etc). Lastly, a **flowchart** can help you design each of your tasks before you begin coding. Doing so helps ensure that all of your code is in the right place and can make it easier to see simple refactorings or places where the code needs to better handle the "unhappy path".

When reading code, the diagrams can help you understand a codebase better by working in reverse. For each function (or, at least, the main tasks) drawing a **flowchart** can help you understand *what* each function is actually trying to do. By seeing the structure of the loops and conditionals, it can be easier to identify the purpose of each chunk of code. Armed with an understanding of what each function does, you can then draw the **block diagram** for the system by identifying how and where the tasks are communicating with each other. Armed with those diagrams, you'll have a roadmap for what the code is actually accomplishing and how it's accomplishing it, making it much easier to understand the impact of each individual line of code.

If you've stuck around this long, thanks for reading, and I hope I was able to share something useful with you! Are you ready for one last challenge? Identify a piece of code that you're working on currently and make a block diagram and flowcharts for it. Or, if you have those already, take what you've seen today and try to improve your block diagram and/or flowcharts in some small way. Happy hacking!